
Backtracking

The Complete Guide

Master backtracking in one day for 42 Exam 03.

Contents

- 01 What is Backtracking?
- 02 The Universal Structure
- 03 Order of Execution
- 04 The Decision Tree
- 05 Common Points
- 06 Things to Watch Out For
- 07 The Universal Template in C
- 08 N-Queens — Full Example

Prepared for 42 students · 2025

Section 01

What is Backtracking?

Backtracking = **Try, Check, Undo.**

It's a brute-force strategy that is **smart about quitting early**. Instead of generating ALL possible combinations and checking them at the end, you build the solution **one piece at a time** and **abandon a path the moment it can't lead to a valid solution**.

Think of it like navigating a maze:

```
START
|
v
[A] --- [B] --- [DEAD END]
|           ^
|           | you BACKTRACK here
|
+--- [C] --- [D] --- [SOLUTION!]
```

You go forward. You hit a wall. You go BACK to the last fork. You try another path. Repeat until you find the exit (or exhaust all paths).

Section 02

The Universal Structure

Every single backtracking problem follows this skeleton. Memorize it.

Pseudocode

```
function backtrack(state, depth):  
  
    // --- STEP 1: BASE CASE ---  
    if depth == target:  
        record/print the solution  
        return  
  
    // --- STEP 2: LOOP OVER CHOICES ---  
    for each choice in available_choices:  
  
        // --- STEP 3: CHECK VALIDITY ---  
        if is_valid(choice, state):  
  
            // --- STEP 4: MAKE THE CHOICE (place) ---  
            apply(choice, state)  
  
            // --- STEP 5: RECURSE (go deeper) ---  
            backtrack(state, depth + 1)  
  
            // --- STEP 6: UNDO THE CHOICE (backtrack!) ---  
            undo(choice, state)
```

The 6 Sacred Steps

Step	Name	What it does
1	Base case	"Am I done?" — stops recursion
2	Loop	"What can I try?" — iterates over candidates
3	Validate	"Is this choice legal?" — prunes bad branches
4	Apply	"Place the piece" — modify state
5	Recurse	"Go deeper" — solve the next sub-problem
6	Undo	"Take back the piece" — restore state for next try

The Golden Rule: Step 4 and Step 6 are ALWAYS mirrors of each other. Whatever you do in Step 4, you UNDO in Step 6. No exceptions.

Section 03

Order of Execution

This is **critical**. Most confusion comes from not understanding the execution order. Let's trace a tiny example: placing numbers 1–3 without repetition.

```
backtrack([], depth=0)
|
|-- try 1: valid? YES -> place 1 -> state=[1]
|
|   |-- try 1: valid? NO(already used)
|   |-- try 2: valid? YES -> place 2 -> state=[1,2]
|   |
|   |   |-- try 1: valid? NO
|   |   |-- try 2: valid? NO
|   |   |-- try 3: valid? YES -> place 3 -> state=[1,2,3]
|   |   |
|   |   |   |-- depth == 3 -> SOLUTION FOUND! print [1,2,3]
|   |   |   |
|   |   |   +-- return (go back up)
|   |   |
|   |   +-- UNDO 3 -> state=[1,2]      <-- BACKTRACK!
|   |
|   +-- UNDO 2 -> state=[1]          <-- BACKTRACK!
|
|-- try 3: valid? YES -> place 3 -> state=[1,3]
|
|   |-- try 2: valid? YES -> place 2 -> state=[1,3,2]
|   |
|   |   |-- depth == 3 -> SOLUTION! print [1,3,2]
|   |   |
|   |   +-- ...
|
+-- UNDO 1 -> state=[]
|
|-- try 2: ... (continues exploring)
```

The Key Insight: The Call Stack IS Your Memory

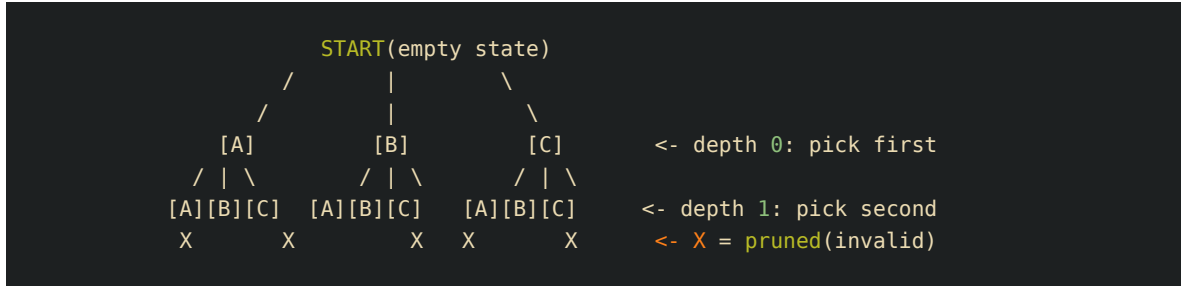
Call Stack(grows down)	What's happening
=====	=====
backtrack(depth=0)	trying choices for position 0
backtrack(depth=1)	trying choices for position 1
backtrack(depth=2)	trying choices for position 2
BASE CASE HIT!	-> print solution
RETURN to depth=2	-> UNDO last choice at depth 2
try next choice	-> or return if no more choices
RETURN to depth=1	-> UNDO last choice at depth 1

When a recursive call RETURNS, execution continues RIGHT AFTER that call. That's where the UNDO step lives. This is how backtracking "remembers" to clean up.

Section 04

The Decision Tree

Every backtracking algorithm explores a **tree of decisions**:



Without backtracking (brute force): you visit ALL leaves. With backtracking: you **cut entire subtrees** when a choice is invalid.

Pruning in Action

```
          START
        /   |   \
      [1]  [2]  [3]
     / | \
  [1] [2] [3]
  X   |   |
    [3] [2]
     V   V
  [1,2,3] [1,3,2]  <- only valid leaves reached!
```

The **X** means: "1 is already placed, don't even bother exploring that subtree." That's **pruning** — what makes backtracking faster than brute force.

Section 05

Common Points

Every backtracking problem you'll encounter at 42 shares these traits.

There's always a state you're building

Problem	State
N-Queens	Board with queens placed so far
Sudoku	Grid with numbers filled so far
Maze solving	Path taken so far
Permutations	Sequence built so far
Subset sum	Elements selected so far

There's always a constraint to check

Problem	Constraint
N-Queens	No two queens attack each other
Sudoku	No duplicate in row, column, or 3x3 box
Maze solving	Can't walk through walls or revisit cells
Permutations	Can't reuse an element
Subset sum	Sum must not exceed target

There's always a way to measure progress

Usually a [depth](#) or [index](#) parameter that increments with each recursive call. When it reaches the target, you've found a solution.

The pattern is always: Place Recurse Remove

```
place()      // modify state
recurse()    // go deeper
remove()     // restore state  <-- THIS IS BACKTRACKING
```

If you forget the REMOVE step, your state gets corrupted and everything breaks.

Section 06

Things to Watch Out For

Always undo what you did

```
// BAD - forgot to undo
if (is_valid(x, y))
{
    board[x][y] = 1;
    solve(depth + 1);
    // oops, board[x][y] is STILL 1!
}

// GOOD
if (is_valid(x, y))
{
    board[x][y] = 1;
    solve(depth + 1);
    board[x][y] = 0;    // <-- UNDO!
}
```

Base case before the loop

If you put the base case after the loop or inside it, you'll either miss solutions or infinite loop.

```
// CORRECT ORDER:
function backtrack(depth):
    if (base_case)          // <-- FIRST!
        return
    while (choices)         // <-- SECOND
        ...

// WRONG ORDER:
function backtrack(depth):
    while (choices)         // <-- loops before checking!
        if (base_case)     // <-- might never reach this
            return
```

Don't confuse depth with the loop variable

```
// WRONG: incrementing depth in the loop
while (c < n)
{
    solve(depth++);    // depth changes each iteration!
}

// CORRECT: depth is fixed for this call
while (c < n)
{
    solve(depth + 1);  // always the same value + 1
    c++;
}
```

Validate before recursing, not after

```
// WRONG: recurse then check
solve(depth + 1);
if (!is_valid())
    undo();

// CORRECT: check then recurse
if (is_valid())
{
    apply();
    solve(depth + 1);
    undo();
}
```

Section 07

The Universal Template in C

This is the skeleton you'll reuse for **every** problem. Only `is_valid()`, `apply()`, and `undo()` change.

```
void solve(int depth)
{
    int choice;

    // Step 1: Base case
    if (depth == target)
    {
        print_solution();
        return ;
    }
    // Step 2: Try each choice
    choice = 0;
    while (choice < num_choices)
    {
        // Step 3: Validate
        if (is_valid(depth, choice))
        {
            // Step 4: Apply
            apply(depth, choice);

            // Step 5: Recurse
            solve(depth + 1);

            // Step 6: Undo
            undo(depth, choice);
        }
        choice++;
    }
}
```

That's it. **Every** backtracking problem at 42 is just this template with different implementations of `is_valid()`, `apply()`, and `undo()`.

Section 08

N-Queens — Full Example

Place N queens on an N×N chessboard so no two queens attack each other (same row, column, or diagonal).

Mapping to the template

Template	N-Queens
state	<code>int queens[] — queens[row] = col</code>
depth	row (which row we're placing in)
target	n (total number of queens/rows)
choices	col = 0 to n-1
<code>is_valid()</code>	check column & diagonal conflicts
<code>apply()</code>	<code>queens[row] = col</code>
<code>recurse()</code>	<code>solve(queens, row + 1, n)</code>
<code>undo()</code>	<code>queens[row] = -1</code>

The Complete Code

Split into individual functions — each one kept together on the page.

```

#include <unistd.h>

int is_valid(int *queens, int row, int col)
{
    int i = 0;
    while (i < row)
    {
        if (queens[i] == col)
            return (0);
        if (queens[i] - col == i - row
            || queens[i] - col == row - i)
            return (0);
        i++;
    }
    return (1);
}

void print_board(int *queens, int n)
{
    int row = 0;
    while (row < n)
    {
        int col = 0;
        while (col < n)
        {
            if (queens[row] == col)
                write(1, "Q", 1);
            else
                write(1, ".", 1);
            if (col < n - 1)
                write(1, " ", 1);
            col++;
        }
        write(1, "\n", 1);
        row++;
    }
    write(1, "\n", 1);
}

```

```

void solve(int *queens, int row, int n)
{
    int col;

    if (row == n) // BASE CASE
    {
        print_board(queens, n);
        return ;
    }
    col = 0;
    while (col < n) // LOOP CHOICES
    {
        if (is_valid(queens, row, col)) // VALIDATE
        {
            queens[row] = col; // APPLY
            solve(queens, row + 1, n); // RECURSE
            queens[row] = -1; // UNDO
        }
        col++;
    }
}

int main(int argc, char **argv)
{
    int n, queens[100], i = 0;

    if (argc != 2)
        return (1);
    n = 0;
    while (argv[1][i] >= '0' && argv[1][i] <= '9')
    {
        n = n * 10 + (argv[1][i] - '0');
        i++;
    }
    if (n <= 0 || n > 100)
        return (1);
    i = 0;
    while (i < n)
        queens[i++] = -1;
    solve(queens, 0, n);
    return (0);
}

```

Section

Quick Reference

```
+-----+
|               BACKTRACKING CHEAT SHEET               |
+-----+

| void backtrack(state, depth)                          |
| {                                                      |
|     if (depth == target)    <- 1. BASE CASE          |
|     { record; return; }    |
|                               |
|     for (each choice)      <- 2. LOOP CHOICES        |
|     {                                                              |
|         if (is_valid(choice)) <- 3. VALIDATE          |
|         {                                                              |
|             apply(choice);    <- 4. PLACE              |
|             backtrack(d + 1); <- 5. RECURSE            |
|             undo(choice);     <- 6. UNDO!!             |
|         }                                                              |
|     }                                                              |
| }                                                                  |
|                                                                 |
| REMEMBER:                                                    |
| - Base case FIRST, before the loop                          |
| - Validate BEFORE recursing                                |
| - ALWAYS undo after recursing                              |
| - Apply and Undo are MIRROR operations                     |
| - depth != loop variable                                    |
|                                                                 |
+-----+
```

Exam Strategy

1. Read the problem. Identify the STATE and the CONSTRAINTS.
2. Write `is_valid()` first — that's where the problem-specific logic lives.
3. Copy the template. Literally use the 6-step skeleton.
4. Adapt three things: `is_valid()`, `apply/undo`, and `print`.
5. Test with small input ($n=1$, $n=4$ for queens).
6. Trust the recursion. Don't think 5 levels deep — make ONE level correct.

"To understand recursion, you must first understand recursion."

But seriously — trust that the recursive call does its job. Your only job is to make one level correct.

Good luck! You've got this.